

Relatório Técnico — Controlador de Vending Machine

SystemVerilog · Synopsys VCS + Design Compiler

Residência em Microeletrônica CI-Expert — Trilha RTL Design

Anderson Soares

1. Repositório

Código-fonte completo (RTL, testbench e síntese):

https://github.com/andersonalexsandro/grupo_andersonsoares_vending

2. Diagrama de blocos

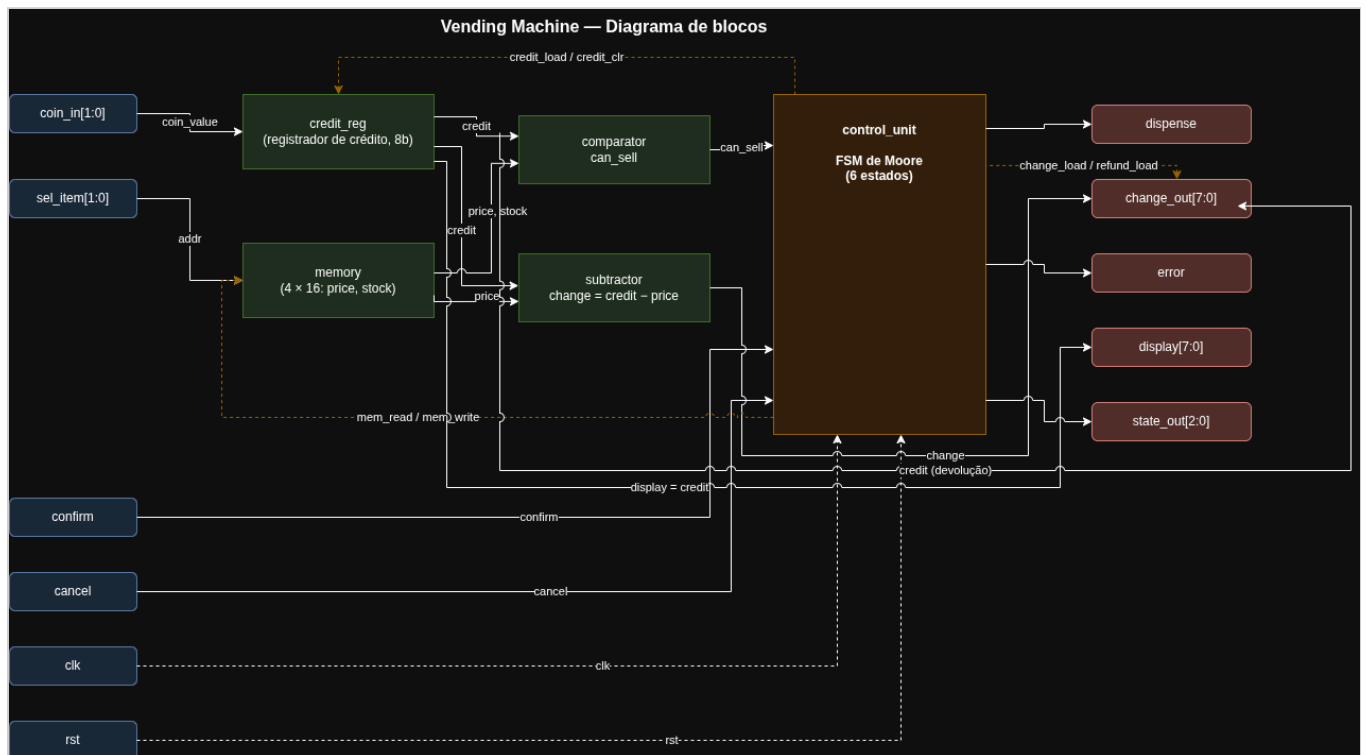


Figura 1 — Diagrama de blocos: entradas → credit_reg, memory, comparator, subtractor, control_unit (FSM) → saídas.

3. Diagrama de estados

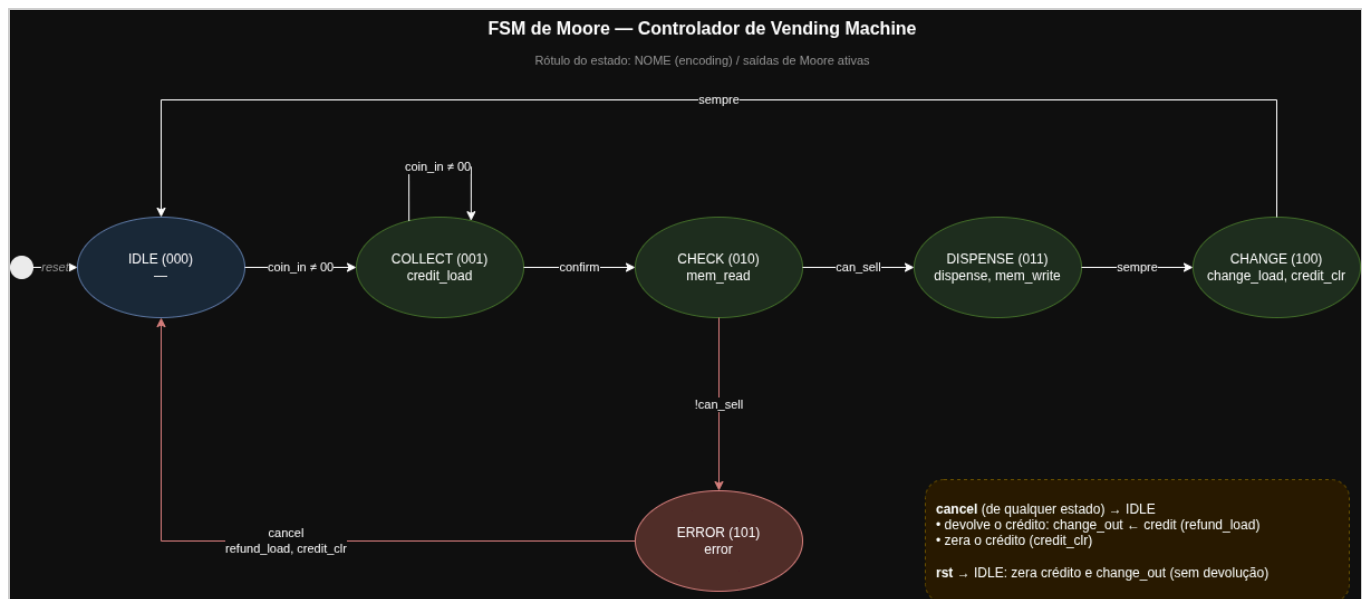


Figura 2 — Diagrama de estados: FSM de Moore, 6 estados (IDLE, COLLECT, CHECK, DISPENSE, CHANGE, ERROR).

4. Descrição dos módulos e decisões de projeto

Módulo	Função
vending_pkg.sv	Package: tipo state_t (6 estados)
credit_reg.sv	Registrador de crédito (8 b, síncrono)
memory.sv	Memória 4×16 (preço + estoque), leitura/escrita síncronas
comparator.sv	Combinacional: can_sell = (credit ≥ price) && (stock > 0)
subtractor.sv	Combinacional: change = credit - price
control_unit.sv	FSM de Moore (6 estados)
vending_top.sv	Top-level: integra os módulos

4.1 credit_clr separado de rst

O crédito precisa ser zerado em duas situações normais de operação: após a conclusão de uma venda, no estado CHANGE, e quando ocorre um cancelamento. Essas situações não representam um reset global da máquina.

Utilizar o sinal rst para zerar o crédito faria com que toda a FSM e os demais registradores fossem reinicializados a cada venda ou cancelamento. Por isso, foi criado o sinal credit_clr, responsável exclusivamente por limpar o registrador de crédito, sem interferir no estado da máquina ou em outros componentes do sistema.

Essa separação mantém uma divisão clara de responsabilidades: rst reinicializa todo o sistema, enquanto credit_clr atua apenas sobre o crédito acumulado.

4.2 mem_read acionado na transição de COLLECT para CHECK

A memória utilizada possui leitura síncrona com latência de um ciclo. Isso significa que os valores de preço e estoque não ficam disponíveis no mesmo ciclo em que a leitura é solicitada.

Caso o sinal mem_read fosse acionado apenas dentro do estado CHECK, os dados de price e stock chegariam somente no ciclo seguinte. Nesse momento, a FSM já estaria tentando decidir se deve seguir para DISPENSE ou ERROR, tornando os dados indisponíveis no instante necessário.

Por esse motivo, a leitura é disparada ainda durante a transição de COLLECT para CHECK, quando o sinal confirm é recebido. Assim, ao entrar no estado CHECK, os dados da memória já estão válidos e o sinal can_sell pode ser calculado corretamente para determinar a próxima transição.

4.3 change_out com duas fontes de carregamento

O registrador change_out precisa atender a duas situações diferentes. Em uma venda concluída normalmente, no estado CHANGE, sua saída deve receber o valor do troco (change_out = credit - price). Já em caso de cancelamento, o valor devolvido deve corresponder ao crédito total inserido (change_out = credit).

O resultado do subtrator, credit - price, não pode ser utilizado para a devolução total. Por isso, o registrador change_out possui duas fontes de carregamento: change_load, utilizada para armazenar o troco, e refund_load, utilizada para armazenar a devolução integral. Quando os dois sinais são considerados, refund_load possui prioridade, garantindo que, em uma operação de cancelamento, todo o crédito seja devolvido corretamente.

4.4 dispense e error: decodificação do estado com gating por cancel

Os sinais dispense e error são decodificados a partir do estado atual da FSM — dispense em DISPENSE e error em ERROR. Há, porém, uma ressalva importante: como o ramo de cancel é avaliado antes do case de estados e zera todas as saídas de controle, a lógica efetiva é dispense = (state == DISPENSE) & ~cancel e error = (state == ERROR) & ~cancel. Ou seja, um cancel em qualquer estado força essas saídas a 0 imediatamente — elas dependem também da entrada cancel, e não apenas do estado.

Como o estado DISPENSE dura exatamente um ciclo, o dispense é naturalmente um pulso de um ciclo, sem necessidade de registrador adicional. Esse gating combinacional por cancel é justamente o que aparece na análise de timing (Seção 8): o caminho crítico vai da entrada cancel até a saída dispense — um caminho entrada → saída que só existe porque dispense depende diretamente de cancel.

5. Waveforms anotadas

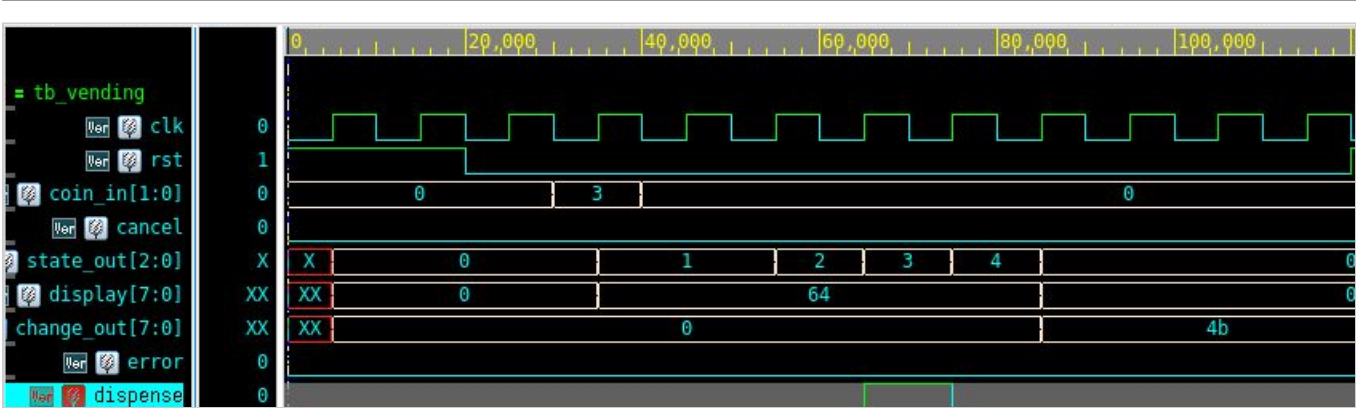


Figura 3 — Cenário 1 (compra com troco): coin_in=R\$1 → display=100; passa por DISPENSE (pulso de dispense); change_out=75; crédito zera.

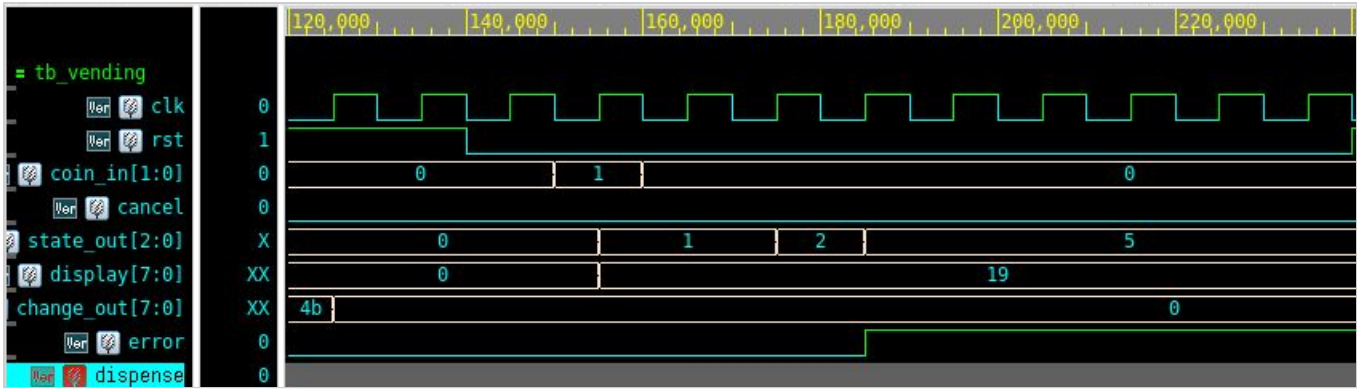


Figura 4 — Cenário 2 (crédito insuficiente): coin_in=R\$0,25 → display=25; snack custa 100 → state=ERROR, error=1.

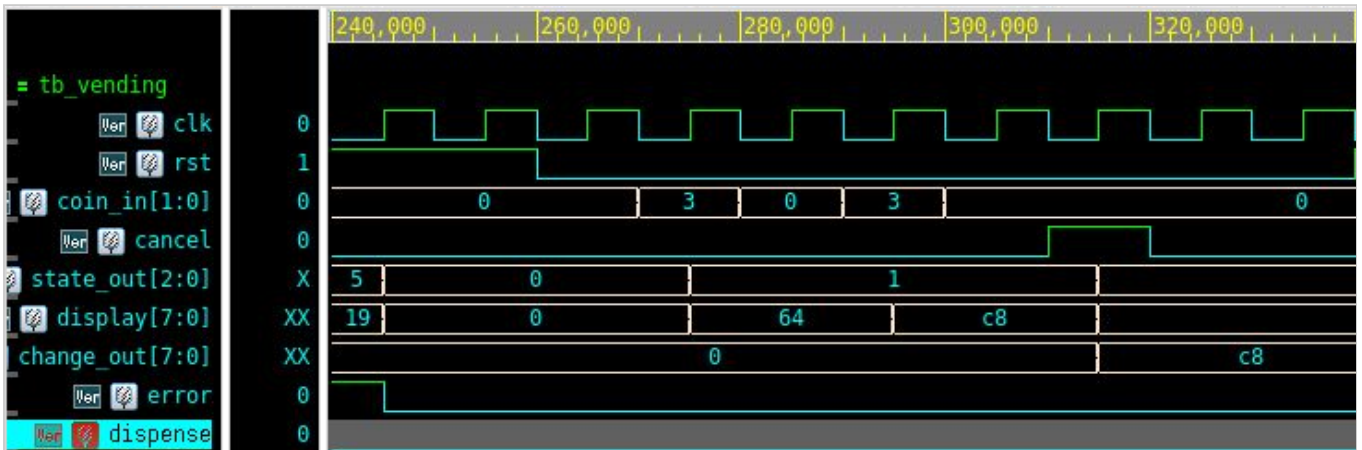


Figura 5 — Cenário 3 (cancelamento): display 100 → 200; cancel=1 → change_out=200 (devolução total), crédito zero, volta a IDLE.

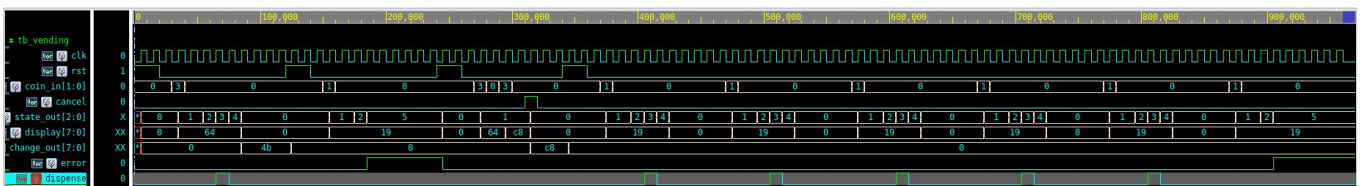


Figura 6 — Cenário 4 (estoque zerado): 5 compras de café (dispense 5×; stock 5 → 0); na 6ª tentativa error=1.

6. Saída do testbench (PASS/FAIL)

Testbench self-checking com os 4 cenários obrigatórios e timeout. Resultado idêntico no Icarus (local) e no VCS (servidor):

```

=== Cenário 1: compra com troco ===
PASS: C1 dispensou / C1 troco=75 / C1 credito=0
=== Cenário 2: credito insuficiente ===
PASS: C2 erro / C2 estado=ERROR
=== Cenário 3: cancelamento ===
PASS: C3 devolucao=200 / C3 credito=0 / C3 volta IDLE
=== Cenário 4: estoque zerado ===
PASS: C4 compra 1..5 / C4 6a compra = erro
>>> TODOS OS TESTES PASSARAM <<< (14/14)

```

7. Resultados de síntese

Biblioteca: SAED32 RVT, corner tt (1,05 V, 25 °C) — saed32rvt_tt1p05v25c.db. No ponto de operação pedido (50 MHz / 20 ns): slack **+13,25 ns (MET)**, área total ≈ **790**.

Exploração de timing (varredura de período):

Período (ns)	Freq (MHz)	Slack (ns)	Área	Status
20	50,0	+13,25	790,1	MET
16	62,5	+9,25	798,3	MET
12	83,3	+5,25	798,3	MET
10	100,0	+3,25	798,3	MET
8	125,0	+1,25	798,3	MET
6	166,7	-0,75	798,3	VIOLATED
5	200,0	-1,75	867,1	VIOLATED
4	250,0	-2,70	1070,7	VIOLATED

$F_{max} \approx 125$ MHz (menor período testado com slack $\geq 0 = 8$ ns; virada real ~ 7 ns).

8. Análise do caminho crítico

8.1 Caminho crítico

O relatório gerado pelo comando `report_timing` indicou que o caminho crítico parte da entrada `cancel`, atravessa a lógica combinacional da FSM e termina na saída `dispense`. Portanto, trata-se de um caminho do tipo entrada-saída (input-to-output), e não de um caminho entre registradores. Em outras palavras, o atraso analisado não corresponde à propagação entre dois elementos sequenciais, mas ao tempo necessário para que uma mudança na entrada `cancel` produza efeito na saída `dispense`.

8.2 Variação da área com a redução do período

Enquanto a restrição de temporização ainda era atendida, a área do circuito permaneceu praticamente estável, variando aproximadamente entre 790 e 798 unidades. Ao impor períodos mais agressivos e incompatíveis com o caminho existente, como 5 ns e 4 ns, o Design Compiler aumentou significativamente a área para 867 e 1071 unidades, respectivamente. Esse crescimento ocorre porque a ferramenta tenta substituir células, reestruturar a lógica e utilizar implementações mais rápidas para reduzir o atraso — mesmo assim, nessas condições, o circuito não conseguiu atender à temporização.

8.3 Frequência máxima de operação

Com período de 8 ns (125 MHz), o circuito ainda atende à restrição, com slack positivo de aproximadamente +1,25 ns. Já com período de 6 ns (~ 167 MHz), ocorre violação, com slack de aproximadamente -0,75 ns. Estima-se que o menor período possível esteja próximo de 7 ns, o que corresponde a uma frequência máxima de aproximadamente 143 MHz.

8.4 Estado e transição envolvidos

O caminho crítico envolve a entrada `cancel` e a saída `dispense`, associada ao estado `DISPENSE` da FSM. Apesar disso, o atraso não é causado principalmente pela complexidade da lógica de estados: os atrasos de entrada e saída definidos nas restrições representam cerca de 3 ns na entrada e 3 ns na saída, enquanto a lógica combinacional interna contribui com apenas $\sim 0,13$ ns. Assim, o tempo total do caminho é dominado pelas premissas de temporização da interface de I/O, e não pela lógica interna da FSM.

8.5 Possível otimização no RTL

Neste caso, uma alteração direta na lógica RTL teria pouco impacto, pois o gargalo principal não está na implementação da FSM, mas nos atrasos de entrada e saída assumidos durante a síntese. A principal forma de alterar o resultado seria revisar a modelagem dos sinais de I/O (`input delay` e `output delay`), desde que isso represente corretamente o ambiente externo. Caso fosse necessário reduzir efetivamente o caminho combinacional entre entrada e

saída, uma possível solução seria registrar a entrada `cancel` e/ou a saída `dispense`, dividindo o caminho em trechos sequenciais menores sincronizados pelo clock — o que, porém, adicionaria latência e deveria ser avaliado conforme os requisitos funcionais.

9. Conclusão

9.1 Compatibilidade entre simuladores

O VCS mostrou-se mais rigoroso do que o Icarus Verilog. Durante a validação, três situações aceitas pelo simulador local foram rejeitadas pelo VCS: (a) a diretiva `timescale` precisava estar definida de forma uniforme em todos os módulos, evitando o erro ITSFM; (b) uma memória inicializada por meio de um bloco `initial` não poderia ser descrita com `always_ff` (erro ICPD), sendo necessário utilizar um bloco `always`; (c) o uso de valores de um tipo `enum` em um operador ternário exigia conversão explícita de tipo, resolvido substituindo o ternário por uma estrutura `if/else`. A principal conclusão é que a validação final deve ser realizada no VCS: simuladores mais leves são úteis durante o desenvolvimento, mas não devem ser considerados suficientes para garantir compatibilidade com o fluxo oficial.

9.2 Latência da memória síncrona

Como a leitura da memória possui latência de um ciclo, os dados solicitados não ficam disponíveis imediatamente. Para garantir que o preço e o estoque estivessem válidos no estado CHECK, o sinal `mem_read` passou a ser acionado ainda na transição de COLLECT para CHECK. Dessa forma, os dados chegam no ciclo correto e podem ser utilizados pela lógica de decisão da FSM.

9.3 Restrições de síntese no arquivo SDC

Durante a síntese, identificou-se que o comando `read_sdc` não aceitava a utilização de `remove_from_collection` da forma inicialmente empregada. Para contornar essa limitação, as portas de entrada e saída precisaram ser listadas explicitamente no arquivo `.sdc`, tornando as restrições mais diretas e compatíveis com o Design Compiler.

9.4 Fluxo Synopsys no servidor

A execução das ferramentas Synopsys no servidor exigiu atenção ao ambiente: carregar as variáveis com `source snps.sh`, verificar a licença, utilizar o X2Go com o ambiente gráfico XFCE e encerrar corretamente a sessão ao finalizar o trabalho, em vez de apenas suspendê-la. Esses cuidados foram importantes para evitar problemas de inicialização, licença e permanência de sessões abertas.

9.5 Lições conceituais de projeto

A separação entre o datapath e a unidade de controle tornou o sistema mais organizado e facilitou a implementação da FSM de Moore: a lógica de controle ficou responsável pelas transições de estado e pelos sinais de comando, enquanto o datapath concentrou operações como armazenamento do crédito, cálculo do troco e acesso à memória.

O uso de um testbench self-checking, com mensagens automáticas de PASS e FAIL e um mecanismo de timeout, permitiu identificar erros ainda nas primeiras etapas, reduzindo a dependência de inspeção manual das formas de onda e tornando a validação mais confiável.

A varredura de períodos durante a síntese evidenciou o trade-off entre área e velocidade: para atender a restrições mais agressivas, a ferramenta aumentou a área do circuito na tentativa de usar estruturas mais rápidas, mesmo sem conseguir fechar o timing nos casos extremos.

Por fim, o projeto mostrou que, em circuitos pequenos, o caminho crítico nem sempre está na lógica interna: neste caso, os atrasos associados à interface de entrada e saída tiveram impacto maior do que a própria lógica combinacional da FSM, destacando a importância de analisar as restrições de I/O e não apenas o RTL ao avaliar o desempenho de um circuito.